

분기 • 파이썬의 비교 연산자 이해하기

https://gemini.google.com/app/10ca3a6151071dd9

User prompt: 1. 파이썬에서 다른 타입의 연산을 비교 연산("==" 등) 하면 어떻게 되는가? 2. is 와 "=="을 비교하시오.

Response: 1. 다른 타입 간의 == 비교

파이썬에서 서로 다른 타입을 == 연산자로 비교하면 **에러(TypeError)**가 발생하지 않고 기본적으로 **False**를 반환합니다. 파이썬의 == 연산자는 객체의 '값(Value)'이 같은지 평가하도록 설계되어 있기 때문입니다.

- 예외 (숫자형 데이터): int, float, complex, bool 등 숫자 형태를 띠는 데이터는 타입이 달라도 수학적 값이 동일하면 True를 반환합니다. (파이썬에서 True는 1, False는 0으로 취급됩니다.)

```
print("1" == 1)           # False (문자열과 정수)
print([1, 2] == (1, 2))   # False (리스트와 튜플)

print(1 == 1.0)           # True (정수와 실수 - 수학적 값이 같음)
print(True == 1)          # True (bool형과 정수)
print(False == 0.0)       # True (bool형과 실수)
```

2. is 와 == 의 비교

두 연산자는 비교하는 기준이 완전히 다릅니다. ==는 데이터의 **내용**을 비교하고, is는 메모리 주소(객체 자체)를 비교합니다.

연산자	비교 기준	내부 동작	설명
==	값 (Value) 동등성	a.__eq__(b)	두 객체에 담긴 내용물(데이터)이 같은지 확인합니다.
is	식별자 (Identity) 동등성	id(a) == id(b)	두 변수가 메모리 상의 정확히 같은 객체 를 가리키는지 확인합니다.

동작 예시:

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

# == (값 비교)
print(a == b) # True (리스트 안의 내용물이 완전히 동일함)
print(a == c) # True (내용물이 동일함)

# is (메모리 주소 비교)
```

```
print(a is b) # False (값은 같지만, 메모리에 각각 따로 생성된 별개의 리스트임)
print(a is c) # True (c는 a와 메모리 상의 완전히 동일한 객체를 가리킴)
```

코딩 관례 (PEP 8): None, True, False와 같은 파이썬의 고유한 싱글톤(Singleton) 객체를 비교할 때는 == 대신 is 연산자를 사용하는 것이 표준입니다. (예: if result is None:)

User prompt: ```@app.get("/models/{model_name}") async def get_model(model_name: ModelName):
 if model_name is ModelName.alexnet: # ModelName Enum 클래스의 멤버 변수명으로 입력값과 비교하거나
 return {"model_name": model_name, "message": f"Deep Learning FTW!", type(model_name)={str(type(model_name))}, type(ModelName.alexnet)={str(type(ModelName.alexnet))}} # {"model_name": "alexnet", "message": "Deep Learning FTW!", type(model_name)=<enum 'ModelName'>, type(ModelName.alexnet)=<enum 'ModelName'>}"
 if model_name.value == "lenet": # 입력된 모델의 value 멤버 변수를 문자열과 비교할 수 있다.
 return {"model_name": model_name, "message": f"LeCNN all the images, type(model_name.value)={str(type(model_name.value))}, type(\\"lenet\\")={str(type(\\"lenet\\"))}} # {"model_name": "lenet", "message": "LeCNN all the images, type(model_name.value)=<class 'str'>, type(\\"lenet\\")=<class 'str'>}"``` 그러한 맥락에서 이 코드를 자세히 설명해줘.

Response: 제공해주신 FastAPI 코드는 앞서 설명해 드린 is (메모리 주소 비교)와 == (값 비교)의 차이를 실무적으로 완벽하게 보여주는 예시입니다.

파이썬의 Enum(열거형) 클래스의 특성과 연결하여 두 방식이 어떻게 동작하는지 자세히 설명해 드리겠습니다.

1. if model_name is ModelName.alexnet: (객체 식별자 비교)

이 부분에서는 == 대신 is를 사용했습니다. 이것이 가능하고 또 권장되는 이유는 파이썬에서 **Enum 멤버는 메모리 상에 단 하나만 존재하는 싱글톤(Singleton) 객체**이기 때문입니다.

- **동작 원리:** FastAPI는 사용자가 URL로 /models/alexnet을 요청하면, 내부적으로 문자열 "alexnet"을 ModelName.alexnet이라는 객체로 변환(Parsing)하여 model_name 변수에 담아줍니다.
- **왜 is를 쓰는가?** 이 객체는 새로 생성된 복사본이 아니라, 클래스를 정의할 때 메모리에 이미 만들어진 바로 그 ModelName.alexnet 객체 자체입니다. 따라서 id(model_name) == id(ModelName.alexnet)가 성립하므로 is 연산자가 True를 반환합니다.
- **결론:** 이전 답변에서 None, True, False 같은 싱글톤 객체는 is로 비교하는 것이 관례라고 말씀드렸는데, **Enum 멤버 역시 싱글톤이므로 is로 비교하는 것이 가장 빠르고 파이썬스러운 (Pythonic) 작성법**입니다.

2. if model_name.value == "lenet": (값 비교)

이 부분에서는 `is`가 아닌 `==`를 사용했습니다. 비교하는 대상이 Enum 객체 자체가 아니라, 객체 안에 들어있는 문자열 값(Value)이기 때문입니다.

- **동작 원리:** `model_name.value`는 열거형 객체에서 실제 값(문자열)을 추출합니다. 주석에 적어주신 대로 이 값의 타입은 `<class 'str'>`입니다. 비교 대상인 `"lenet"` 역시 `<class 'str'>`입니다.
- **왜 `==`를 쓰는가?** 두 문자열의 '**내용**'이 일치하는지 확인해야 하므로 값의 동등성을 평가하는 `==`를 사용해야 합니다. (이곳에 `is`를 사용하면 문자열 메모리 할당 방식에 따라 버그가 발생할 위험이 있습니다.)
- **결론:** 순수하게 데이터의 내용(Value)을 비교할 때는 예외 없이 `==`를 사용해야 한다는 원칙을 잘 따르고 있습니다.

💡 (심화) 만약 `model_name == "lenet"` 이라고 비교한다면? (질문 1번과 연결)

앞선 질문에서 "다른 타입의 연산을 `==`로 비교하면 어떻게 되는가?"라고 물어보셨습니다. 이 코드에서 `.value`를 빼고 비교해보면 그 특징이 나타납니다.

```
if model_name == "lenet":
```

- `model_name`의 타입은 `<enum 'ModelName'>`입니다.
- `"lenet"`의 타입은 `<class 'str'>`입니다.
- 일반적인 파이썬 Enum을 사용했다면, **타입이 다르기 때문에 값도 다르다고 판단하여 예러 없이 `False`를 반환합니다.**
- **참고:** FastAPI에서는 보통 `class ModelName(str, Enum):` 처럼 `str`을 상속받아 Enum을 정의합니다. 이렇게 `str`을 상속받은 Enum(또는 파이썬 3.11 이상의 `StrEnum`)이라면 예외적으로 파이썬 내부에서 문자열로 취급할 수 있어 `True`가 나올 수도 있습니다. 하지만 `.value`를 명시적으로 꺼내어 비교하거나, `is`를 통해 객체 자체를 비교하는 것이 훨씬 명확하고 안전한 코드입니다.

User prompt: 서로 다른 클래스간 비교가 가능하게 만드는 방법이 있어? 만약 이러한 것을 인위적으로 작업해놓지 않은 상태에서 비교하면 어떻게 돼?

Response: 네, 파이썬에서는 매직 메서드(Magic Method)를 오버라이딩(Overriding)하여 전혀 다른 클래스 간에도 비교가 가능하도록 직접 정의할 수 있습니다.

어떤 작업도 해두지 않은 기본 상태에서 일어나는 일부터, 직접 비교 가능하게 만드는 방법까지 순서대로 설명해 드리겠습니다.

1. 인위적인 작업이 없는 상태에서의 비교 (기본 동작)

서로 다른 클래스의 인스턴스를 아무런 설정 없이 비교하면 연산자의 종류에 따라 결과가 다르게 나타납니다.

- **동등 비교 (==, !=): 에러 없이 False 반환** 파이썬의 모든 클래스는 기본적으로 최상위 object 클래스를 상속받습니다. 별도의 설정을 하지 않으면 객체의 내부 데이터가 아닌 메모리 주소 (Identity)를 비교하게 됩니다. 서로 다른 클래스의 인스턴스는 당연히 다른 메모리에 생성되므로 무조건 False가 나옵니다.
- **크기 비교 (<, >, <=, >=): TypeError 발생** 크기를 비교하는 연산은 기본 동작이 정의되어 있지 않습니다. 따라서 "두 객체 간에 해당 연산을 지원하지 않는다(not supported)"는 내용의 TypeError 에러가 발생하며 프로그램이 멈춥니다.

```
class Dog:
    def __init__(self, weight):
        self.weight = weight
```

```
class Cat:
    def __init__(self, weight):
        self.weight = weight
```

```
dog = Dog(5)
cat = Cat(5)
```

1. 동등 비교

```
print(dog == cat) # False (에러는 나지 않지만, 주소가 달라 무조건 False)
```

2. 크기 비교

```
# print(dog > cat) # TypeError: '>' not supported between instances of 'Dog' and 'Cat'
```

2. 다른 클래스 간 비교 가능하게 만드는 방법 (__eq__ 매직 메서드)

클래스 내부에 `__eq__(self, other)` 라는 특수한 메서드를 정의하면 `==` 연산자가 호출될 때의 동작을 내 마음대로 조작할 수 있습니다. 이때 `isinstance()`를 사용해 상대방(other)의 타입을 확인한 후, 비교할 논리를 작성하면 됩니다.

실제 실무에서 자주 쓰이는 온도 단위 변환(섭씨와 화씨)을 예시로 들겠습니다.

```
class Fahrenheit:
    def __init__(self, temp):
        self.temp = temp
```

```
class Celsius:
    def __init__(self, temp):
        self.temp = temp
```

'=' 연산자가 사용될 때 호출되는 메서드

```
def __eq__(self, other):
```

1. 비교 대상이 Fahrenheit 클래스인 경우

```
if isinstance(other, Fahrenheit):
```

화씨를 섭씨로 변환하여 값이 같은지 비교

```
converted_temp = (other.temp - 32) * 5 / 9
```

```
return self.temp == converted_temp
```

2. 비교 대상이 같은 Celsius 클래스인 경우

```
elif isinstance(other, Celsius):
```

```
return self.temp == other.temp
```

```
# 3. 그 외의 알 수 없는 타입인 경우 기본 동작으로 넘김  
return NotImplemented
```

```
# 테스트
```

```
celsius = Celsius(10)          # 섭씨 10도  
fahrenheit = Fahrenheit(50)    # 화씨 50도 (섭씨로 10도)
```

```
# 전혀 다른 두 클래스를 == 로 비교  
print(celsius == fahrenheit)  # True
```

다른 비교 연산자를 위한 매직 메서드 == 외에 다른 연산자들도 각각의 매직 메서드를 클래스 안에 정의해 주면 서로 다른 클래스 간에 크기 비교도 가능해집니다.

- `__eq__` : == (Equal)
 - `__ne__` : != (Not Equal)
 - `__lt__` : < (Less Than)
 - `__le__` : <= (Less Than or Equal)
 - `__gt__` : > (Greater Than)
 - `__ge__` : >= (Greater Than or Equal)
-

User prompt: 서로 다른 클래스끼리 비교하는데, 둘이 "`__eq__`"가 다르면 어떻게 돼?

Response: 파이썬은 두 객체를 ==로 비교할 때 '**왼쪽 객체**'에게 우선권을 주며, 왼쪽 객체가 비교를 포기하면 '**오른쪽 객체**'에게 기회를 넘기는 **릴레이 방식**으로 동작합니다.

두 클래스의 `__eq__` 로직이 다를 때 파이썬 내부에서 일어나는 과정은 크게 두 가지 상황으로 나눌 수 있습니다.

1. 한쪽이 비교 방법을 몰라 포기하는 경우 (NotImplemented 반환) 일반적으로 잘 짜여진 `__eq__` 메서드는 자신이 모르는 타입이 들어오면 에러를 내는 대신 파이썬의 특수 상수인 `NotImplemented`를 반환합니다.

- `A == B`를 실행하면 파이썬은 먼저 `A.__eq__(B)`를 호출합니다.
- A가 "나는 B와 비교하는 법을 몰라"라며 `NotImplemented`를 반환하면, 파이썬은 포기하지 않고 순서를 뒤집어 오른쪽 객체인 `B.__eq__(A)`를 호출합니다.
- B가 비교 방법을 알아서 `True`나 `False`를 반환하면 그 값이 최종 결과가 됩니다.
- 만약 B마저 `NotImplemented`를 반환하면, 파이썬은 두 객체가 다르다고 판단하여 최종적으로 `False`를 반환합니다.

2. 양쪽 다 비교 로직이 있지만, 서로 결과가 다른 경우 (논리 충돌) A는 B와 같다고 주장하는데, B는 A와 다르다고 주장하는 경우입니다. 이때는 **연산자 왼쪽에 있는 객체의 `__eq__` 결과가 무조건 최종 승리**합니다.

```
class StubbornA:
    def __eq__(self, other):
        print("A의 기준: 무조건 같다!")
        return True # 상대가 누구든 True 반환

class StubbornB:
    def __eq__(self, other):
        print("B의 기준: 무조건 다르다!")
        return False # 상대가 누구든 False 반환

a = StubbornA()
b = StubbornB()
```

```
# 1. a가 왼쪽에 있을 때
print(a == b)
# 출력: A의 기준: 무조건 같다! -> True (B의 __eq__는 아예 호출되지 않음)
```

```
# 2. b가 왼쪽에 있을 때
print(b == a)
# 출력: B의 기준: 무조건 다르다! -> False (A의 __eq__는 아예 호출되지 않음)
```

이처럼 비교하는 순서에 따라 결과가 달라지는 것을 '**비대칭적(Asymmetric) 비교**'라고 부릅니다. 파이썬 문법상으로는 전혀 문제가 없지만, $a == b$ 와 $b == a$ 의 결과가 다르면 개발자가 심각한 혼란을 겪을 수 있으므로 실무에서는 양쪽의 논리가 일치하도록 설계하는 것이 원칙입니다.

예외 규칙 (자식 클래스 우선의 법칙): 만약 오른쪽 객체(B)가 왼쪽 객체(A)를 상속받은 ****자식 클래스(Subclass)****라면, 파이썬은 왼쪽/오른쪽 위치와 상관없이 자식 클래스인 **`B.__eq__(A)`를 무조건 먼저 호출**합니다. 부모의 낡은 비교 로직보다 자식의 최신(오버라이딩된) 비교 로직이 더 정확할 것이라고 파이썬이 가정하기 때문입니다.
